

Entity States in Entity Framework Core (EF Core)

In this article, I will discuss **Entity States in Entity Framework Core (EF Core)**. Please read our previous article discussing [CRUD Operations in Entity Framework Core](#). We will work with the same example we have worked on so far.

Entity States in Entity Framework Core (EF Core)

In Entity Framework Core (EF Core), entities can exist in various states during their lifecycle as the DbContext tracks them. Understanding these entity states is essential for correctly managing data and understanding how EF Core interacts with the database. Entity States in EF Core represent an entity's state in relation to its interaction with the DbContext. EF Core uses these states to determine what operations need to be performed on the database when `SaveChanges()` is called.

So, each entity tracked by the DbContext has a state that indicates how the entity should be processed during the `SaveChanges()` method call. The entity state is represented by an enum called `EntityType` in EF Core with the following signature.

```
namespace Microsoft.EntityFrameworkCore
{
    public enum EntityState
    {
        Detached,
        Unchanged,
        Deleted,
        Modified,
        Added
    }
}
```

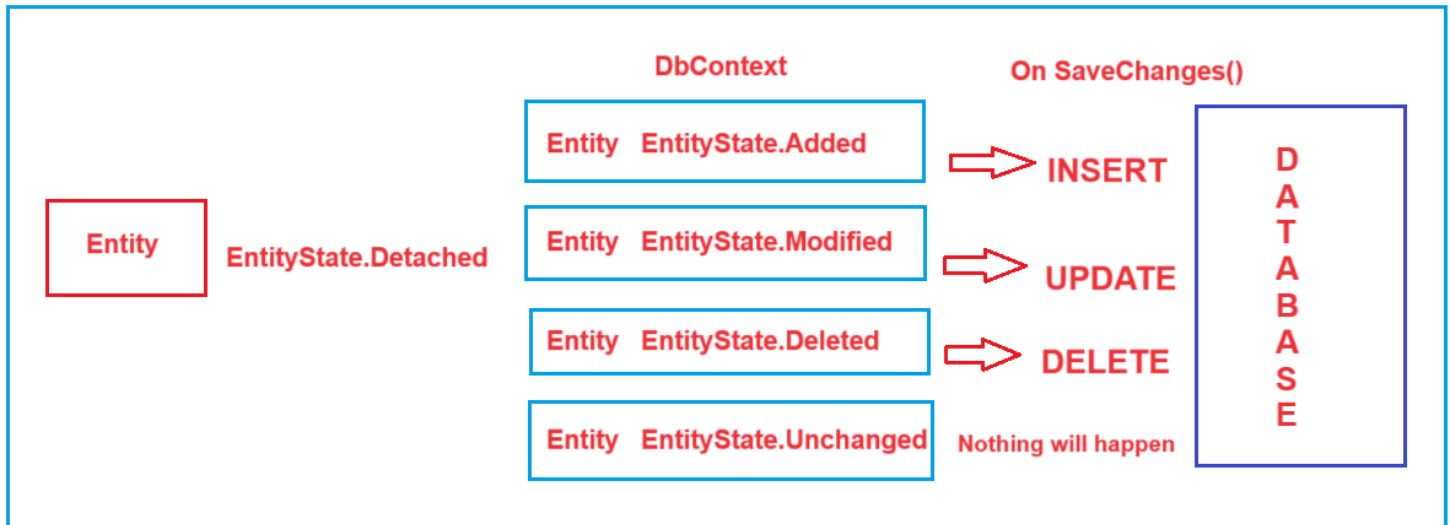
As you can see, there are five states of an entity, and an entity is always in any one of the above states. Let us proceed and understand these states in detail:

- **Added:** The entity is being tracked by the context but does not yet exist in the database. When `SaveChanges()` is executed, EF Core issues an INSERT command for this entity.
- **Unchanged:** The entity is being tracked and exists in the database but has not been modified since it was retrieved. EF Core will not issue any database commands for entities in this state during `SaveChanges()`.
- **Modified:** The entity is being tracked and exists in the database, and some or all of its property values have been modified. When `SaveChanges()` is executed, EF Core issues a UPDATE command that only updates the changed properties.
- **Deleted:** The entity is being tracked and exists in the database but has been marked for deletion. When `SaveChanges()` is executed, EF Core issues a DELETE command for this entity.
- **Detached:** The entity is not being tracked by the context. EF Core will not perform any database operations on detached entities when `SaveChanges()` is called. This state is typical for entities that have been created but not yet added to the context, have been explicitly detached from the context, or have been deleted from the database.

Entity Lifecycle in Entity Framework Core

When working with Entity Framework Core (EF Core), it's essential to understand the entity lifecycle clearly. The entity lifecycle involves different states and transitions that determine how entities interact with the DbContext and, ultimately, to the database.

The change in the entity state from the Unchanged to the Modified state is the only state automatically handled by the context class. All other changes must be made explicitly using the proper DbContext class methods. The following diagram shows the different states of an entity in the Entity Framework.



So, the Context object not only holds the reference to all the entity objects it retrieved from the database but also tracks entity states and maintains modifications to the entity's properties. This feature is known as **Change Tracking**. So, let us proceed and understand Change Tracking EF Core.

What is Change Tracking in EF Core?

Change Tracking is a mechanism in EF Core that tracks changes made to entities after they are retrieved from the database or added to the context. When the DbContext tracks an entity, EF Core monitors any changes made to its property values. This allows EF Core to know what operations (insert, update, delete) need to be performed on the database when **SaveChanges()** is called.

When an entity is first loaded into the DbContext or attached to it, EF Core creates a snapshot of the entity's original values. This snapshot is stored internally and is later used to detect changes by comparing the current values of the entity with the original snapshot.

Types of Change Tracking in EF Core

EF Core supports different types of change tracking, which can be categorized based on the level of tracking and how they are applied:

- **Automatic Change Tracking:** When entities are retrieved from the database or added to the context, EF Core automatically tracks changes to their properties. This is the default behavior for most operations in EF Core. It is ideal for scenarios where we need to persist database changes and need EF Core to handle the tracking automatically.
- **No-Tracking Queries:** In scenarios where we only read data and do not intend to modify or persist changes, you can use No-Tracking queries. This improves performance because EF Core does not spend resources on tracking changes. It is suitable for read-only operations where data retrieval is the primary focus. For example:
- **Explicit Change Tracking:** In some cases, you might want to control the change tracking process manually. EF Core allows you to explicitly tell the DbContext when an entity should be tracked, modified, or detached. It is useful when you need complete control over how and when entities are tracked.

Note: Before proceeding, first, delete all the data from the database tables by executing the following commands:

```
-- Truncate the Foreign Key Table
TRUNCATE TABLE Students;
GO
-- Delete All Records from the Primary Key Table
DELETE FROM Branches;
GO
-- RESEED The Identity
DBCC CHECKIDENT ('EFCoreDB1.dbo.Branches', RESEED, 0);
```

EF Core Added State Example

Whenever we add a new Entity to the context object using the Add method of DbSet or DbContext, then the state of the entity will be in the Added state. Added entity state indicates that the entity exists in the context but does not exist in the database. In this case, DbContext generates the INSERT SQL Statement and inserts the data into the database when the SaveChanges method is invoked. Once the SaveChanges method execution is successful, the state of the Entity is changed from Added to Unchanged state.

Let's consider a simple scenario in which we have a Student entity in a school management system. The system allows new student registration, and when a new student registers, the application adds the student's information to the database. For a better understanding, please modify the Program class as follows. The following example code is self-explained, so please read the comment lines for a better understanding.

```
using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            try
            {
                using (var context = new EFCoreDbContext())
                {
                    // Create a new Branch
                    var branch = new Branch
                    {
                        BranchName = "Computer Science",
                        Description = "Computer Science Department",
                        PhoneNumber = "123-456-7890",
                        Email = "cs@example.com"
                    };
                    // Create a new Student
                    var student = new Student
                    {
                        FirstName = "John",
                        LastName = "Doe",
                        DateOfBirth = new DateTime(2000, 1, 1),
                        Gender = "Male",
                        Email = "john.doe@example.com",
                        PhoneNumber = "555-555-5555",
                        EnrollmentDate = DateTime.Now,
                    };
                }
            }
        }
    }
}
```

```

        Branch = branch
    };
    // Display the Student Entity state before adding to the
context
        Console.WriteLine($"Student Entity State before adding to
the context: {context.Entry(student).State}");
        // Add the student to the context
        // Using DbSet Add Method
        context.Students.Add(student);
        // Using DbContext Add Method
        // context.Add(student);
        // Display the Student Entity state after adding to the
context
        Console.WriteLine($"Student Entity State after adding to
the context: {context.Entry(student).State} \n");
        // Save changes to the database
        // This will save both Branch and Student entity to the
database

        context.SaveChanges();
        // Display the Student Entity state after saving changes
        Console.WriteLine($"Student Entity State after saving
changes: {context.Entry(student).State}");
    }
}
catch (DbUpdateException dbEx)
{
    Console.WriteLine($"Database update error: {dbEx.Message}");
}
catch (Exception ex)
{
    Console.WriteLine($"An error occurred: {ex.Message}");
}
}
}
}

```

Output:

```

Student Entity State before adding to the context: Detached
Student Entity State after adding to the context: Added

info: 02-09-2024 10:51:34.578 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (20ms) [Parameters=[@p0='?' (Size = 4000), @p1='?' (Size = 4000), @p2='?' (Size = 4000), @p3='?'
      (Size = 4000)], CommandType='Text', CommandTimeout='30']
      SET IMPLICIT_TRANSACTIONS OFF;
      SET NOCOUNT ON;
      INSERT INTO [Branches] ([BranchName], [Description], [Email], [PhoneNumber])
      OUTPUT INSERTED.[BranchId]
      VALUES (@p0, @p1, @p2, @p3);
info: 02-09-2024 10:51:34.595 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (2ms) [Parameters=[@p4='?' (DbType = Int32), @p5='?' (DbType = DateTime2), @p6='?' (Size = 4000
      ), @p7='?' (DbType = DateTime2), @p8='?' (Size = 4000), @p9='?' (Size = 4000), @p10='?' (Size = 4000), @p11='?' (Size =
      4000)], CommandType='Text', CommandTimeout='30']
      SET IMPLICIT_TRANSACTIONS OFF;
      SET NOCOUNT ON;
      INSERT INTO [Students] ([BranchId], [DateOfBirth], [Email], [EnrollmentDate], [FirstName], [Gender], [LastName], [
      PhoneNumber])
      OUTPUT INSERTED.[StudentId]
      VALUES (@p4, @p5, @p6, @p7, @p8, @p9, @p10, @p11);
↓ INSERT SQL Statement
Student Entity State after saving changes: Unchanged

```

If you verify the database, you will see the above entity is being added to the Students database table, as shown below. Please note the Student ID, which we will use in our following examples.

```
SELECT * FROM Students;
```

StudentId	FirstName	LastName	DateOfBirth	Gender	Email	PhoneNumber	EnrollmentDate	BranchId
1	John	Doe	2000-01-01 ...	Male	john.doe@example.com	555-555-5555	2024-09-02 10...	8

Unchanged State of an Entity in Entity Framework Core with Example

In Entity Framework Core (EF Core), the Unchanged state refers to an entity that EF Core tracks but recognizes as identical to what is currently stored in the database. When an entity is in an Unchanged state, it means that no modifications have been made to the entity since it was retrieved from the database or since the last time `SaveChanges()` was called.

When we call the `SaveChanges()` method, EF Core ignores entities in the Unchanged state, meaning no operations are performed on these entities in the database. This is the default state for entities retrieved from a database query or attached to the context using the `Attach()` method.

Let's consider a scenario where we have a Student entity in a school management system, and we want to retrieve a student's information to display it on a webpage without making any modifications. To better understand, please modify the Program class as follows. The following example code is self-explanatory, so please read the comments.

```

using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            try
            {
                using (var context = new EFCoreDbContext())

```

```

{
    // Retrieve the Student with StudentId 1 from the
database
    var student = context.Students.Find(1);
    if (student == null)
    {
        Console.WriteLine("Student with ID 1 not found.");
        return;
    }
    // Display the state of the student after retrieval
    Console.WriteLine($"Entity State after retrieval:
{context.Entry(student).State}");
    // Simulate calling SaveChanges without modifying the
entity
    context.SaveChanges();
    Console.WriteLine("SaveChanges called. Since the entity
was in the Unchanged state, no operations were performed on the database.");
    Console.WriteLine($"Entity State after SaveChanges:
{context.Entry(student).State}");
}
}
catch (DbUpdateException dbEx)
{
    Console.WriteLine($"Database update error: {dbEx.Message}");
}
catch (Exception ex)
{
    Console.WriteLine($"An error occurred: {ex.Message}");
}
}
}
}

```

Output:

```
info: 02-09-2024 11:06:03.270 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (28ms) [Parameters=[@__p_0='?' (DbType = Int32)], CommandType='Text', CommandTimeout='30']
      SELECT TOP(1) [s].[StudentId], [s].[BranchId], [s].[DateOfBirth], [s].[Email], [s].[EnrollmentDate], [s].[FirstName], [s].[Gender], [s].[LastName], [s].[PhoneNumber]
      FROM [Students] AS [s]
      WHERE [s].[StudentId] = @__p_0
Entity State after retrieval: Unchanged
SaveChanges called. Since the entity was in the Unchanged state, no operations were performed on the database.
Entity State after SaveChanges: Unchanged
```

Attaching an Existing Entity to the Context Object

Imagine a system where a background job processes a list of student updates. The updates might come from an external source, such as a file upload or an API call. This background job needs to update multiple student records efficiently. However, since the entities come from an external source, they are initially not tracked by the EF Core context.

In such cases, we can use the Attach method to bring these entities into the context, mark only the properties that need updating as Modified, and then save the changes. This allows EF Core to generate the appropriate UPDATE SQL statements without loading the original entities from the database.

Please modify the Program class as follows. In the below example, we demonstrate how to update specific properties of multiple entities in a disconnected scenario using Entity Framework Core (EF Core) while understanding and observing the various entity states (Detached, Unchanged, Modified, Unchanged) throughout the process. The following example code is self-explained, so please read the comment line.

```
using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            try
            {
                // Simulating a list of student phone number updates received
                from an external source
                var studentUpdates = new List<Student>
                {
                    //Currently we have only one entity in the database with
                    Id 1
                    new Student { StudentId = 1, PhoneNumber = "111-111-
                    1111", Email="john.doe@dotnettutorials.com" }
                };
                using (var context = new EFCoreDbContext())
                {
                    foreach (var updatedStudent in studentUpdates)
                    {
                        // Initial state before attaching (Detached)
                        Console.WriteLine($"Before Attach: StudentId
                        {updatedStudent.StudentId}, State: {context.Entry(updatedStudent).State}");
                        // Attach the student to the context (state should be
                        Unchanged)
                        context.Students.Attach(updatedStudent);
                        Console.WriteLine($"After Attach: StudentId
                        {updatedStudent.StudentId}, State: {context.Entry(updatedStudent).State}");
                        // Mark the PhoneNumber and Email properties as
                        modified (state should be Modified)
                        context.Entry(updatedStudent).Property(s =>
                        s.PhoneNumber).IsModified = true;
                        context.Entry(updatedStudent).Property(s =>
                        s.Email).IsModified = true;
                        Console.WriteLine($"After Marking PhoneNumber as
                        Modified: StudentId {updatedStudent.StudentId}, State:
                        {context.Entry(updatedStudent).State}");
                    }
                    // Save all changes to the database in one batch
                    context.SaveChanges();
                }
            }
        }
    }
}
```

```

        Console.WriteLine("Student Phone numbers and Emails
Updated successfully.");
        // Final state after saving (should be Unchanged)
        foreach (var updatedStudent in studentUpdates)
        {
            Console.WriteLine($"After SaveChanges: StudentId
{updatedStudent.StudentId}, State: {context.Entry(updatedStudent).State}");
        }
    }
}
catch (DbUpdateException dbEx)
{
    Console.WriteLine($"Database update error: {dbEx.Message}");
}
catch (Exception ex)
{
    Console.WriteLine($"An error occurred: {ex.Message}");
}
}
}
}
}

```

Output:

```

Before Attach: StudentId 1, State: Detached
After Attach: StudentId 1, State: Unchanged
After Marking PhoneNumber as Modified: StudentId 1, State: Modified
info: 02-09-2024 11:26:30.914 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database
.Command)
    Executed DbCommand (29ms) [Parameters=[@p2='?' (DbType = Int32), @p0='?' (Size = 4000), @p1='?' (Size =
4000)], CommandType='Text', CommandTimeout='30']
    SET IMPLICIT_TRANSACTIONS OFF;
    SET NOCOUNT ON;
    UPDATE [Students] SET [Email] = @p0, [PhoneNumber] = @p1
    OUTPUT 1
    WHERE [StudentId] = @p2;
Student Phone numbers and Emails Updated successfully.
After SaveChanges: StudentId 1, State: Unchanged

```

Detached State of an Entity in Entity Framework Core with Example

In Entity Framework Core (EF Core), the Detached state refers to an entity state that the DbContext is not tracking. EF Core is unaware of any changes made to this entity state and will not perform any database operations when SaveChanges() is called. An entity can be in the Detached state for several reasons:

- It was never attached to the DbContext (e.g., a new object created in memory).
- It was explicitly detached using the Detach method.
- It was attached, but the context was disposed, making it effectively detached.

Consider a scenario where we are developing an application that processes a large number of entities in a batch operation, such as a data import or a background job that processes and updates thousands of records. Suppose we keep all these entities attached to the DbContext. In that case, EF Core will continue tracking all of them, which can consume a lot of memory and reduce performance, especially in long-running operations.

We can detach entities from the DbContext to optimize performance once they have been processed. This prevents EF Core from tracking them, reducing memory usage and avoiding potential slowdowns due to the DbContext managing a

large number of tracked entities.

We will simulate a scenario where we update students' Email addresses as part of a bulk operation and then detach each Student entity after processing to free up memory and improve performance. Please have a look at the following example for a better understanding. The following example code is self-explained, so please go through the comment line.

```
using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            try
            {
                using (var context = new EFCoreDbContext())
                {
                    // Fetching students from the database who have an email
                    with "dotnettutorials.com" domain
                    var studentsToProcess = context.Students
                                                    .Where(s =>
s.Email.Contains("dotnettutorials.com"))
                                                    .ToList();
                    foreach (var student in studentsToProcess)
                    {
                        // Update the student's Email address by replacing
                        the domain with "example.com"
                        student.Email = ReplaceDomain(student.Email,
"example.com");

                        // Save changes to the database
                        context.SaveChanges();
                        Console.WriteLine($"StudentId {student.StudentId}
email updated to '{student.Email}'");
                        // Detach the student to free up memory after
                        processing
                        context.Entry(student).State = EntityState.Detached;
                        Console.WriteLine($"Detached StudentId
{student.StudentId}, State: {context.Entry(student).State}");
                    }
                    Console.WriteLine("All students processed
successfully.");
                }
            }
            catch (Exception ex)
            {
                Console.WriteLine($"An error occurred: {ex.Message}");
            }
        }
        // Helper method to replace the email domain
    }
}
```

```

static string ReplaceDomain(string email, string newDomain)
{
    var atIndex = email.IndexOf('@');
    if (atIndex >= 0)
    {
        return email.Substring(0, atIndex + 1) + newDomain;
    }
    return email; // In case the email doesn't contain '@', return it
as is
}
}
}

```

Output:

```

info: 02-09-2024 12:02:49.940 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database
.Command)
    Executed DbCommand (25ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
    SELECT [s].[StudentId], [s].[BranchId], [s].[DateOfBirth], [s].[Email], [s].[EnrollmentDate], [s].[First
Name], [s].[Gender], [s].[LastName], [s].[PhoneNumber]
    FROM [Students] AS [s]
    WHERE [s].[Email] LIKE N'%dotnettutorials.com%'
info: 02-09-2024 12:02:50.030 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database
.Command)
    Executed DbCommand (18ms) [Parameters=[@p1='?' (DbType = Int32), @p0='?' (Size = 4000)], CommandType='Te
xt', CommandTimeout='30']
    SET IMPLICIT_TRANSACTIONS OFF;
    SET NOCOUNT ON;
    UPDATE [Students] SET [Email] = @p0
    OUTPUT 1
    WHERE [StudentId] = @p1;
StudentId 1 email updated to 'john.doe@example.com'
Detached StudentId 1, State: Detached
All students processed successfully.

```

The above example shows how to modify the domain part of the students' Email addresses in the database, replacing the old domain "dotnettutorials.com" with "example.com." The changes are persisted in the database, and the entities are detached after processing to optimize memory usage.

Modified State in Entity Framework Core (EF Core) with Example

In Entity Framework Core (EF Core), the Modified state indicates that an entity's properties have been changed since it was retrieved from the database or attached to the DbContext. EF Core tracks these changes when an entity is in the Modified state. It generates the necessary SQL UPDATE statements to persist the modifications to the database when SaveChanges() is called.

Imagine you have an application where students can update their personal information, such as their email address or phone number. When a student submits the updated information, the application retrieves the student's record from the database, applies the updates, and then saves the changes. EF Core will automatically mark the modified properties as Modified, and when SaveChanges() is called, the changes are persisted in the database.

Please have a look at the following example for a better understanding. The following example code is self-explained, so please go through the comment line.

```

using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    class Program

```

```

{
    static void Main(string[] args)
    {
        try
        {
            using (var context = new EFCoreDbContext())
            {
                // Fetch the student from the database
                var student = context.Students.FirstOrDefault(s =>
s.StudentId == 1);
                if (student != null)
                {
                    Console.WriteLine($"Initial State:
{context.Entry(student).State}");
                    // Update the student's phone number
                    student.PhoneNumber = "555-123-4567";
                    // EF Core will automatically mark the entity as
Modified
                    Console.WriteLine($"State after modifying
PhoneNumber: {context.Entry(student).State}");
                    // Save changes to the database
                    context.SaveChanges();
                    Console.WriteLine("Student phone number updated
successfully.");
                    // State after saving changes should be Unchanged
                    Console.WriteLine($"State after SaveChanges:
{context.Entry(student).State}");
                }
                else
                {
                    Console.WriteLine("Student not found.");
                }
            }
        }
        catch (Exception ex)
        {
            Console.WriteLine($"An error occurred: {ex.Message}");
        }
    }
}

```

Output:

```

info: 02-09-2024 13:14:08.524 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database
.Command)
    Executed DbCommand (16ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
    SELECT TOP(1) [s].[StudentId], [s].[BranchId], [s].[DateOfBirth], [s].[Email], [s].[EnrollmentDate], [s]
.[FirstName], [s].[Gender], [s].[LastName], [s].[PhoneNumber]
    FROM [Students] AS [s]
    WHERE [s].[StudentId] = 1
Initial State: Unchanged
State after modifying PhoneNumber: Modified
info: 02-09-2024 13:14:08.725 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database
.Command)
    Executed DbCommand (125ms) [Parameters=[@p1='?' (DbType = Int32), @p0='?' (Size = 4000)], CommandType='T
ext', CommandTimeout='30']
    SET IMPLICIT_TRANSACTIONS OFF;
    SET NOCOUNT ON;
    UPDATE [Students] SET [PhoneNumber] = @p0
    OUTPUT 1
    WHERE [StudentId] = @p1;
Student phone number updated successfully.
State after SaveChanges: Unchanged

```

Note: The Entity framework keeps track of the properties that have been modified. The Columns in the Update statement are set only for those columns whose values have been modified.

Deleted State in EF Core with Example

In Entity Framework Core (EF Core), the Deleted state indicates that an entity is marked for deletion but has not yet been deleted from the database. When an entity is in the Deleted state, EF Core generates a SQL DELETE statement to remove the corresponding record from the database when `SaveChanges()` is called. We can mark an entity as Deleted using the `Remove` method or by explicitly setting the entity's state to Deleted.

If an entity is deleted, related entities (depending on the relationship configuration and cascade delete rules) might also be automatically marked for deletion. Once an entity is marked as Deleted, EF Core tracks it in this state until `SaveChanges()` is called. At this point, it is permanently removed from the database.

Consider an educational application where an administrator can delete a student's record from the system. This could be necessary if a student withdraws from the school or if a record needs to be removed due to an error. When the administrator decides to delete a student, the application marks the student entity as Deleted, and then `SaveChanges()` is called to remove the record from the database.

Consider a scenario where an administrator deletes a student from the system. Please have a look at the following example for a better understanding. The following example code is self-explained, so please read the comment lines for a better understanding.

```

using EFCoreCodeFirstDemo.Entities;
using System.Diagnostics.Metrics;
using System.Diagnostics;
namespace EFCoreCodeFirstDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            try
            {
                using (var context = new EFCoreDbContext())
                {
                    // Fetch the student from the database
                    // When the student is retrieved, its initial state is
                    Unchanged, indicating that no changes have been made.

```

```

        var student = context.Students.FirstOrDefault(s =>
s.StudentId == 1);
        if (student != null)
        {
            Console.WriteLine($"Initial State:
{context.Entry(student).State}");
            //The Remove method is called on the student entity.
            //This method marks the entity as Deleted.
            context.Students.Remove(student);
            //After calling Remove, the entity's state changes to
Deleted, which is tracked by EF Core.
            // The state should now be Deleted
            Console.WriteLine($"State after marking for deletion:
{context.Entry(student).State}");
            // When SaveChanges() is called, EF Core generates a
DELETE SQL statement to remove the student's record from the database.
            // Save changes to the database (this will delete the
record)

            context.SaveChanges();
            //After SaveChanges() completes, the entity is no
longer tracked by the context because it has been deleted.
            Console.WriteLine("Student record deleted
successfully.");
        }
        else
        {
            Console.WriteLine("Student not found.");
        }
    }
}
catch (Exception ex)
{
    Console.WriteLine($"An error occurred: {ex.Message}");
}
}
}
}

```

Output:

```

info: 02-09-2024 13:22:54.031 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database
.Command)
    Executed DbCommand (18ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
    SELECT TOP(1) [s].[StudentId], [s].[BranchId], [s].[DateOfBirth], [s].[Email], [s].[EnrollmentDate], [s]
.[FirstName], [s].[Gender], [s].[LastName], [s].[PhoneNumber]
    FROM [Students] AS [s]
    WHERE [s].[StudentId] = 1
Initial State: Unchanged
State after marking for deletion: Deleted
info: 02-09-2024 13:22:54.134 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database
.Command)
    Executed DbCommand (20ms) [Parameters=[@p0='?' (DbType = Int32)], CommandType='Text', CommandTimeout='30
']
    SET IMPLICIT_TRANSACTIONS OFF;
    SET NOCOUNT ON;
    DELETE FROM [Students]
    OUTPUT 1
    WHERE [StudentId] = @p0;
Student record deleted successfully.

```

What SaveChanges Method Does in Entity Framework Core?

The SaveChanges method is responsible for persisting changes made to entities tracked by the DbContext to the database. Depending on the state of the entities, SaveChanges performs different actions:

Unchanged State:

- If the entities are in an Unchanged state, the SaveChanges method will not touch them. This is because Unchanged entities have not been modified, added, or marked for deletion. Therefore, there is no need to send them to the database.
- The Unchanged state indicates that the entity in memory is identical to the corresponding entity in the database. As a result, EF Core does not generate any SQL statements for these entities, making the SaveChanges process more efficient.

Added State:

- If the entities are in the Added state, the SaveChanges method inserts them into the database by generating and executing an INSERT SQL statement. Once the SaveChanges method executes successfully, these entities are moved from the Added state to the Unchanged state.
- The Added state indicates that the entity is new and does not yet exist in the database. EF Core generates an INSERT statement to add the entity to the database. After insertion, the entity is considered synchronized with the database, so its state is changed to Unchanged.

Modified State:

- If the entities are in the Modified state, the SaveChanges method updates them in the database by generating and executing a UPDATE SQL statement. Once the SaveChanges method executes successfully, the entities are moved from the Modified state to the Unchanged state.
- The Modified state indicates that the entity's properties have been changed compared to the corresponding database record. EF Core generates an UPDATE statement to reflect these changes in the database. After successfully applying the update, the entity is considered synchronized with the database, and its state is set to Unchanged.

Deleted State:

- If the entities are in the Deleted state, the SaveChanges method removes these entities from the database by generating and executing a DELETE SQL statement. Once the SaveChanges method executes successfully, these entities are detached from the context, meaning the DbContext no longer tracks their state.

- The Deleted state indicates that the entity should be removed from the database. EF Core generates a DELETE statement to remove the record. After deletion, the entity is no longer needed by the context, so it is detached, meaning the DbContext stops tracking its state.

In the next article, I will discuss [LINQ to Entities in Entity Framework Core](#). In this article, I explain the Entity States in Entity Framework Core with Examples. I hope you enjoy this article on Entity States in Entity Framework Core.